딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사

목차

A table of Contents

01 지난시간 복습

02 과적합 진단

03 Dropout

04 BatchNorm

05 데이터 증강

06 Early Stopping과 종합 실험

PART 01

지난시간 복습

1. 복습: 실험 기록표와 CNN

실험	바꾼 것	best val (epoch)	test	Baseline 대비 (val)
Baseline	없음, MLP 784-100-10	97.25% (10)	97.34%	기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001	97.43% (10)	97.59%	+0.18%p
실험 2	모델, MLP → CNN	98.54% (9)	98.67%	**+1.29%p**

- 지난시간 CNN은 같은 학습 함수로 Baseline을 넘었고, 사진을 조금 옮겨도 MLP보다 잘 버텼다
- 오늘은 **처음 보는 데이터에서도 맞히는 힘**, 일반화를 높이는 방법을 다룬다

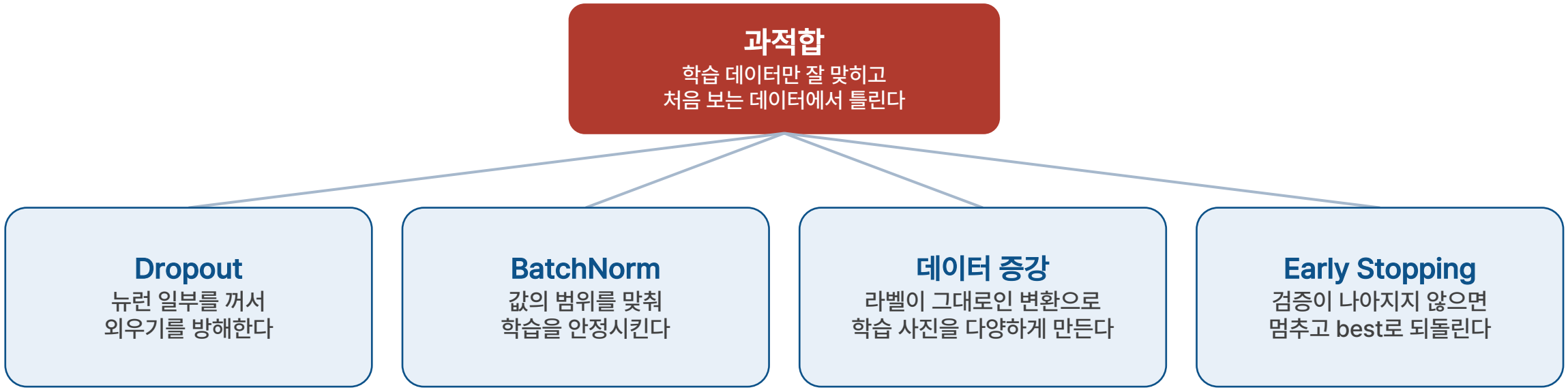
1. 오늘의 질문: 데이터가 적으면 CNN도 외울까

학습 데이터	학습 데이터 정확도	검증 정확도	간격
1,000장	97.30%	88.33%	**8.97%p**
5,000장	98.80%	92.43%	6.37%p
50,000장	99.84%	97.72%	2.12%p

MLP를 30 epoch 학습한 실측

- MLP는 학습 데이터가 적을수록 **간격이 컸다**

1. 과적합을 줄이는 네 가지 도구



- 가장 확실한 대책은 데이터를 더 모으는 것이지만, 늘 가능하지는 않다
- 오늘은 **같은 1,000장**으로 도구를 하나씩 비교하고, 마지막에 Dropout, BatchNorm, 증강 세 가지를 합친다

1. 준비 셀 1: 데이터

준비 셀 1. 50,000장, 10,000장, 그리고 1,000장 로더

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split, Subset
device = "cuda" if torch.cuda.is_available() else "cpu"
torch.backends.cudnn.deterministic = True       # cuDNN도 결정적 계산만
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())
g = torch.Generator().manual_seed(0)            # 학습과 검증 분리, seed 0
train_set, val_set = random_split(train_all, [50000, 10000], generator=g)
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
val_loader = DataLoader(val_set, batch_size=256, shuffle=False)
small_idx = train_set.indices[:1000]            # 새 줄, 학습 데이터 앞 1,000장의 번호
small_loader = DataLoader(Subset(train_all, small_idx), batch_size=64, shuffle=True)
print(len(train_loader), len(val_loader), len(small_loader))   # → 782 40 16
```

- **새로 더한 것**
  - import 줄에 번호로 사진을 고르는 `Subset`을 더했다
  - `train_set.indices`는 50,000장의 원래 번호 목록이고, 앞 1,000개가 `small_idx`다
  - `small_loader`는 그 1,000장만 꺼내는 로더다
- **확인**
  - `782 40 16`이 나오면 성공이다, 1,000장은 배치 64장씩 16개다

1. 준비 셀 2: CNN과 loss

준비 셀 2. 지난시간 CNN

```
class CNN(nn.Module):                          # 지난시간 CNN 그대로
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x):
        x = self.pool(self.act(self.conv1(x)))   # (N, 16, 14, 14)
        x = self.pool(self.act(self.conv2(x)))   # (N, 32, 7, 7)
        return self.fc(self.flat(x))

loss_fn = nn.CrossEntropyLoss()
print(sum(p.numel() for p in CNN().parameters()))   # → 20490
```

- 지난시간 CNN 그대로이고, 파라미터는 **20,490개**다
- 오늘 만드는 모델은 모두 이 클래스를 **복사해 몇 줄만** 바꾼다

1. 준비 셀 3: 검증 함수와 학습 함수

준비 셀 3-1. 검증 함수

```python
def evaluate(model, loader):        # 검증 함수, 그대로
    model.eval()
    total_loss, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            total_loss += loss_fn(out, y).item() * len(y)
            correct += (out.argmax(dim=1) == y).sum().item()
    n = len(loader.dataset)
    return total_loss / n, correct / n * 100
```

준비 셀 3-2. 학습 함수

```python
def train_one_epoch(model, loader, opt):    # 학습 함수, 그대로
    model.train()
    total_loss = 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        loss = loss_fn(model(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()
        total_loss += loss.item() * len(y)
    return total_loss / len(loader.dataset)
```

- 두 함수는 그대로다, 오늘은 `model.eval()`과 `model.train()`이 처음으로 **결과를 바꾼다** (Dropout, BatchNorm)

1. 준비 셀 4: fit, 오늘의 새 표준 틀

준비 셀 4. 학습과 기록을 함수 하나로

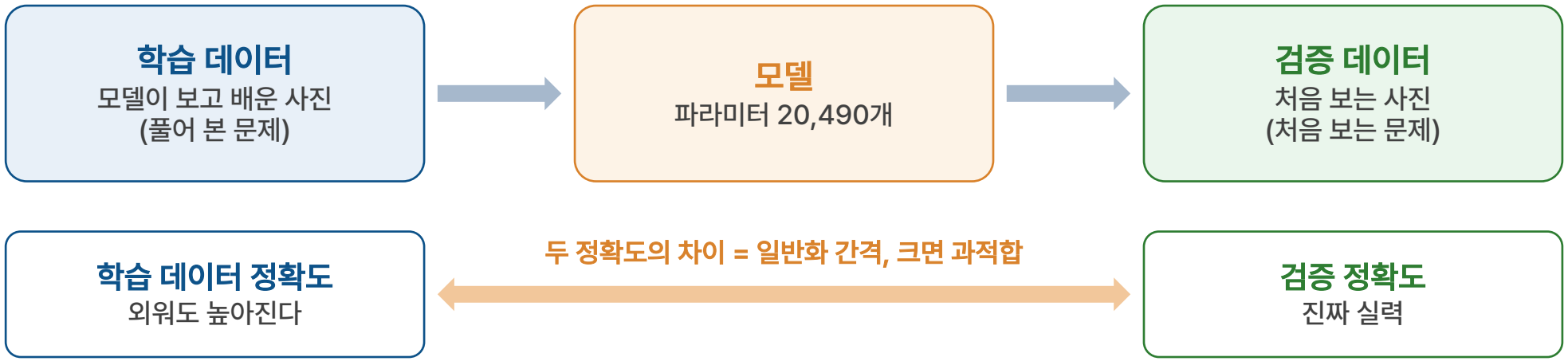
```
def fit(model, loader, epochs=50, opt=None):   # 새 표준 틀
    if opt is None:                            # 따로 주지 않으면 SGD lr 0.1
        opt = torch.optim.SGD(model.parameters(), lr=0.1)
    train_losses, val_losses, val_accs = [], [], []
    best_acc, best_epoch = 0.0, 0
    for epoch in range(epochs):
        tl = train_one_epoch(model, loader, opt)
        vl, va = evaluate(model, val_loader)
        train_losses.append(tl)
        val_losses.append(vl)
        val_accs.append(va)
        if va > best_acc:                      # 최고 기록
            best_acc, best_epoch = va, epoch + 1
    print("best", best_epoch, round(best_acc, 2))
    return train_losses, val_losses, val_accs
```

- **읽는 법**
  - 안쪽은 지금까지 써 온 **epoch 루프**에 **최고 기록 찾기**를 합친 것이다
  - 기록 리스트 세 개를 돌려주고, 끝나면 **best epoch과 검증 정확도**를 한 줄 출력한다
  - best는 **출력만** 하고, 모델은 마지막 epoch의 상태로 남는다
- **쓰는 법**
  - `fit(model, small_loader)`처럼 부르면 1,000장으로 50 epoch 학습한다

PART 02

과적합 진단

2. 일반화와 과적합



일반화 간격 $\text{gap} = A_{\text{train}} - A_{\text{val}}$ A_{train}은 학습이 끝난 뒤 학습 데이터 정확도, A_{val}은 마지막 epoch의 검증 정확도

- **일반화**는 처음 보는 데이터에서도 잘 맞히는 능력이고, **과적합**은 학습 데이터에만 맞춰져 처음 보는 데이터에서 틀리는 상태다
- 간격이 크면 과적합을 의심한다, 판단은 늘 **검증 데이터**로 한다

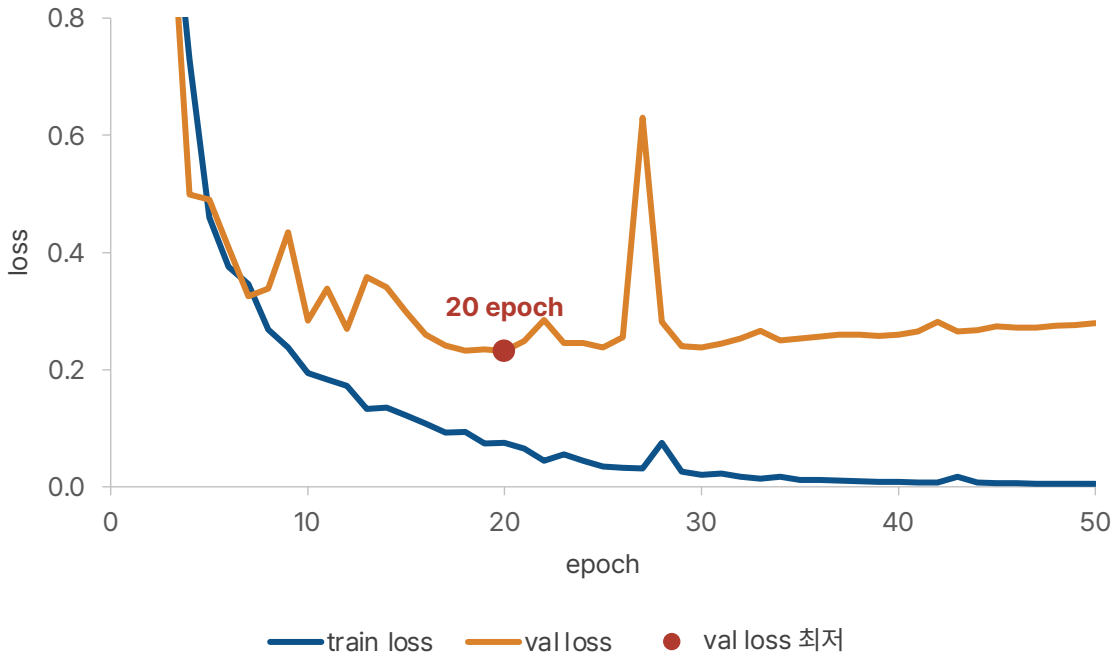
2. 예제 6-1: 1,000장으로 50 epoch 학습

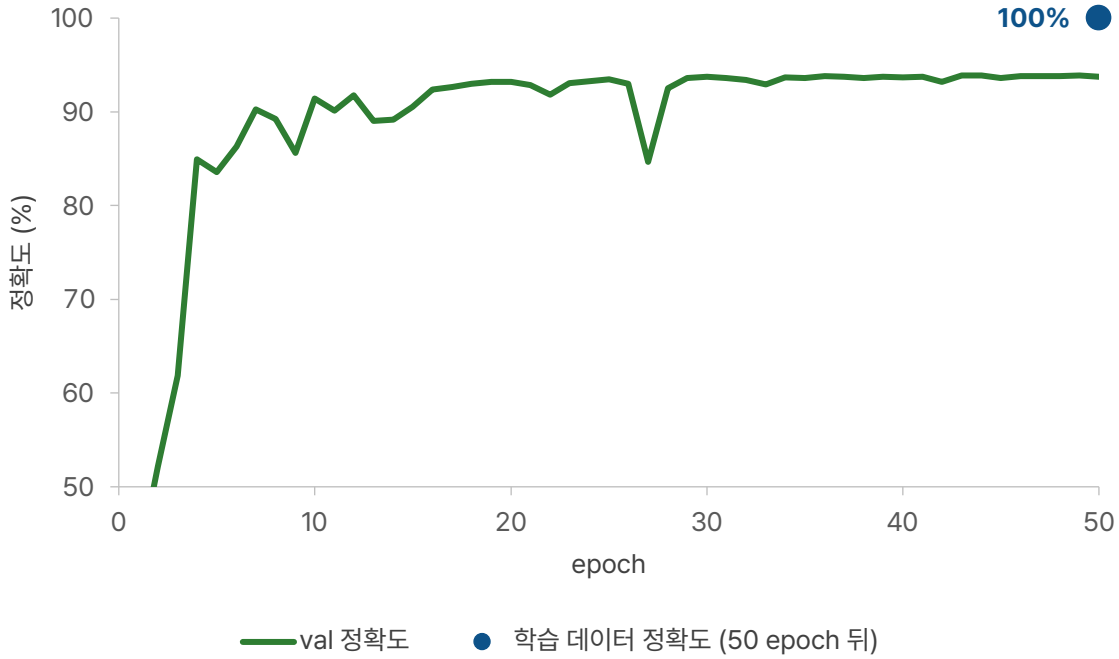
예제 6-1. CNN을 1,000장으로 학습하고 두 정확도 비교

```
torch.manual_seed(0)
model = CNN().to(device)
train_losses, val_losses, val_accs = fit(model, small_loader)   # 1,000장, 50 epoch
# → (예) best 49 93.88
train_loss, train_acc = evaluate(model, small_loader)   # 학습한 1,000장의 정확도
print(round(train_acc, 2), round(val_accs[-1], 2))     # → (예) 100.0 93.73   (학습 데이터 대 검증)
```

- 학습한 1,000장은 **100.0%** 맞히는데 처음 보는 검증 데이터는 **93.73%**라, 간격이 6.27%p다
- CNN도 데이터가 적으면 **외운다**, 오늘은 이 간격을 줄이는 것이 목표다

2. 결과: 학습 곡선에 나타난 과적합

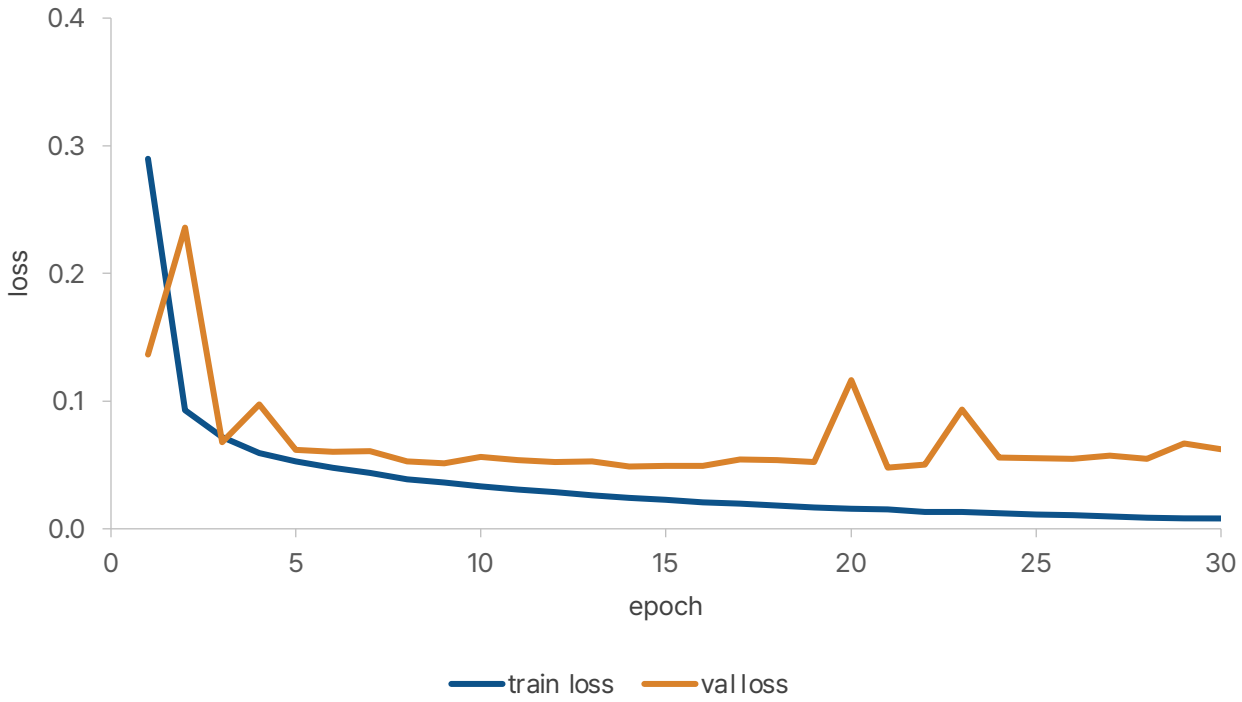




예제 6-1의 기록, CNN 1,000장 50 epoch 실측 (왼쪽은 처음 몇 epoch의 큰 loss를 자르고 0.8까지)

- train loss는 0 근처까지 내려가는데 val loss는 **20 epoch부터 다시 오른다**, 검증 정확도는 30 epoch 이후 거의 오르지 않는다
- 이 모양이 과적합의 전형적인 학습 곡선이다

2. 전체 데이터에서는 간격이 작다



같은 CNN, 전체 50,000장 30 epoch 실측

학습 데이터	학습 데이터 정확도	검증 정확도	간격
1,000장 (50 epoch)	100.00%	93.73%	**6.27%p**
50,000장 (30 epoch)	99.70%	98.52%	**1.18%p**

- 데이터가 많은 쪽은 같은 CNN이라도 간격이 작았다, epoch과 갱신 횟수도 함께 다른 관찰 결과다
- 그래서 대책의 효과는 **데이터가 적을 때** 크게 보인다, 오늘의 비교 실험은 1,000장으로 한다

2. 과적합의 원인 세 가지

원인	1,000장 실험에서	대책 (오늘)	그 밖의 대책
1. 데이터가 적다	사진 1,000장에 파라미터 20,490개	데이터 증강	데이터 더 모으기
2. 모델이 자유롭다	몇몇 뉴런이 특정 사진을 외울 수 있다	Dropout	작은 모델, weight decay (이후)
3. 너무 오래 학습한다	검증이 더 나아지지 않는데도 계속 학습	Early Stopping	best 모델 저장

- BatchNorm은 원래 학습을 안정시키려고 만든 계층인데, 이 실험에서는 간격도 줄였다
- 오늘의 도구는 모두 **처음 보는 데이터의 정확도**를 올리려는 것이다, 학습 데이터 정확도는 오히려 조금 내려갈 수 있다

2. 실습 1: 무작위 정답도 외울 수 있을까 (문제)

실습

1. 아래 셀로 `rand_loader`를 만들고, 예제 6-1을 복사해 로더와 optimizer(Adam)를 바꿔 60 epoch 학습하기
2. 학습 데이터 정확도와 검증 정확도 비교하기
3. 검증 정확도가 왜 그렇게 나왔는지 한 문장으로 쓰기

실습 1. 로더 만들기

```
from torch.utils.data import TensorDataset   # 사진 묶음과 정답 묶음을 짝짓는다
xs = torch.stack([train_all[i][0] for i in small_idx])   # 같은 사진 1,000장을 한 텐서로
torch.manual_seed(0)
ys = torch.randint(0, 10, (1000,))                        # 정답은 0~9 무작위
rand_loader = DataLoader(TensorDataset(xs, ys), batch_size=64, shuffle=True)
print(xs.shape, ys.shape)   # → torch.Size([1000, 1, 28, 28]) torch.Size([1000])
```

- opt는 `torch.optim.Adam(model.parameters(), lr=0.001)`, 학습은 `fit(model, rand_loader, 60, opt)`로 한다
- 학습 데이터 정확도는 `evaluate(model, rand_loader)`로 잰다, 무작위 정답 기준이다

2. 실습 1: 정답 예시

실습 1. 정답 예시

```
torch.manual_seed(0)
model = CNN().to(device)
opt = torch.optim.Adam(model.parameters(), lr=0.001)      # 빨리 외우도록 Adam
train_losses, val_losses, val_accs = fit(model, rand_loader, 60, opt)   # 1. 학습
# → (예) best 9 12.57   (검증은 찍는 수준에 머문다)
train_loss, train_acc = evaluate(model, rand_loader)      # 2. 무작위 정답 기준
print(round(train_acc, 2), round(val_accs[-1], 2))        # → (예) 97.1 9.97
```

- **관찰**
  - 규칙 없는 정답인데도 학습 데이터를 **97.1%** 맞힌다, CNN은 1,000장 정도는 **외울 수 있다**
  - 검증 정확도는 **10.0%**, 외운 무작위 정답은 처음 보는 사진에 쓸모가 없어 10개 중 하나를 찍는 수준이다
- **교훈**
  - 학습 데이터 정확도만으로는 배웠는지 외웠는지 모른다, **처음 보는 데이터**로 확인한다

2. 예제 문제, 학습 곡선 진단

문제

1. 1,000장 CNN에서 val loss는 20 epoch부터 오르는데 검증 정확도는 거의 그대로다, loss만 오르는 이유는?
2. 무작위 정답을 97% 외운 모델과 예제 6-1의 모델, 학습 데이터 정확도만 보면 둘을 구분할 수 있을까?
3. 전체 50,000장에서 간격이 작은데도 Dropout 같은 대책을 1,000장으로 시험하는 이유는?

2. 예제 문제 풀이

문제	풀이	근거 (실측)
1	틀린 사진에 점점 더 확신해 loss가 오른다, 과적합이 시작된 신호	20 → 50 epoch, 틀린 사진의 평균 loss 2.81 → 4.13, 맞힌 사진 쪽은 줄었다
2	구분할 수 없다, 차이는 **검증 정확도**에서만 보인다	학습 데이터 100% 대 97%, 검증 93.7% 대 10.0%
3	간격이 작으면 효과도 작아 흔들림에 묻힌다, 간격이 큰 1,000장에서 비교한다	간격 1.18%p 대 6.27%p

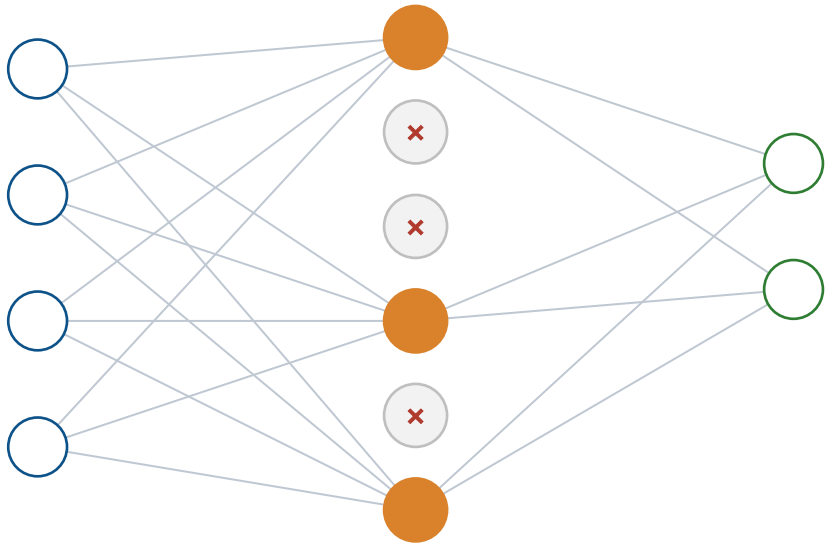
- 학습 데이터 정확도는 외워도 오른다, 판단은 늘 **검증 데이터**로 한다

PART 03

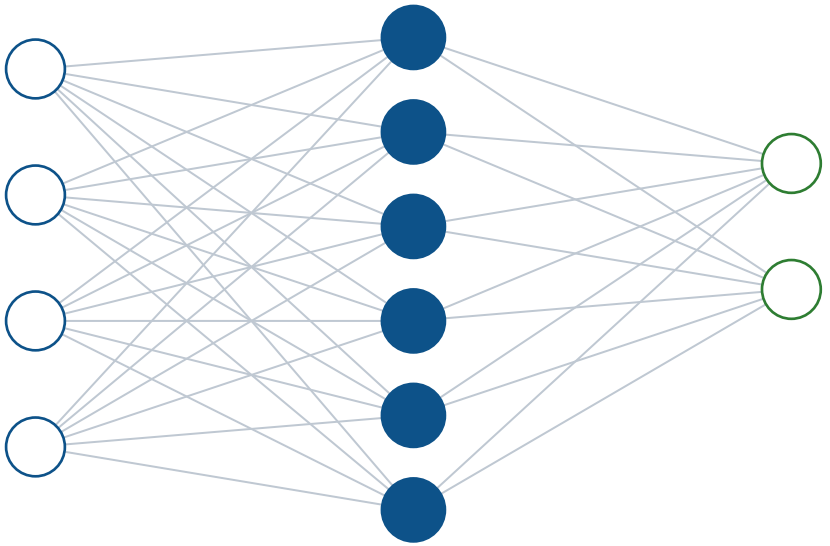
Dropout

3. Dropout의 원리

학습 중, p = 0.5 (step마다 다른 뉴런을 끈다)



평가 중, model.eval() (모두 켠다)



남은 값의 배율 $s = \frac{1}{1-p} = \frac{1}{1-0.5} = 2$ p는 값마다 끌 확률(평균적으로 끄는 비율), 0.5면 남은 값을 2배로 키운다

- 학습할 때 뉴런 출력 중 비율 **p**만큼을 **무작위로 0**으로 만든다, step마다 꺼지는 뉴런이 바뀌어 몇몇 뉴런에 기대어 외우기 어렵다
- 남은 값은 1/(1 – p)배로 키워 여러 번의 평균이 끄기 전과 같게 하고, 평가할 때는 **모두 켠다** (`model.eval()`)

3. 예제 6-2: nn.Dropout이 하는 일

예제 6-2. 학습 모드와 평가 모드에서 Dropout 통과시키기

```
import torch
import torch.nn as nn
torch.manual_seed(0)
drop = nn.Dropout(p=0.5)                # 절반을 무작위로 끈다
x = torch.ones(1, 8)                    # 1로 채운 값 8개
drop.train()                            # 학습 모드
print(drop(x))                          # → (예) tensor([[0., 0., 2., 0., 0., 0., 2., 2.]])
drop.eval()                             # 평가 모드
print(drop(x))                          # → tensor([[1., 1., 1., 1., 1., 1., 1., 1.]])
```

- 학습 모드에서는 절반 정도가 0이 되고, 남은 값은 **2배**(1 ÷ (1 − 0.5))가 된다, `drop(x)`를 다시 부르면 0이 되는 자리가 바뀐다
- 평가 모드에서는 **그대로** 통과한다, 모델 전체에서는 `model.train()`과 `model.eval()`이 이 스위치를 바꾼다

3. 예제 6-3: CNN에 Dropout 넣기

예제 6-3. CNN을 복사해 두 곳 변경

```
class CNNDrop(nn.Module):                    # CNN을 복사해 두 곳만 바꾼다
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.drop = nn.Dropout(0.5)          # 바뀐 곳 1
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x):
        x = self.pool(self.act(self.conv1(x)))
        x = self.pool(self.act(self.conv2(x)))
        return self.fc(self.drop(self.flat(x)))   # 바뀐 곳 2, fc 앞에서 끈다

print(sum(p.numel() for p in CNNDrop().parameters()))   # → 20490   (그대로)
```

- **바뀐 곳**
  - 1. `__init__`에 `nn.Dropout(0.5)` 한 줄을 더했다
  - 2. `forward`에서 펼친 값을 **fc에 넣기 전에** 통과시킨다
- **확인**
  - Dropout에는 학습할 파라미터가 없어 **20,490개 그대로**다

3. 실습 2: Dropout으로 간격 줄이기 (문제)

실습

1. 예제 6-1을 복사해 모델만 `CNNDrop()`으로 바꿔 1,000장으로 학습하기
2. best 검증 정확도와 학습 데이터 정확도를 예제 6-1과 비교하기
3. Dropout 비율을 0.2와 0.8로 바꿔 best 검증 정확도 비교하기

모델	best 검증 정확도 (epoch)	학습 데이터 정확도	마지막 검증 정확도	간격
CNN (예제 6-1)	93.88% (49)	100.00%	93.73%	6.27%p
CNNDrop (실습 2)				

- 같은 seed, 같은 1,000장, 같은 50 epoch으로 **모델만** 바꿔야 공정한 비교다
- 3은 클래스의 `nn.Dropout(0.5)`를 고친 뒤 클래스 셀을 **다시 실행**해야 바뀐다, 끝나면 0.5로 되돌리고 다시 실행한다

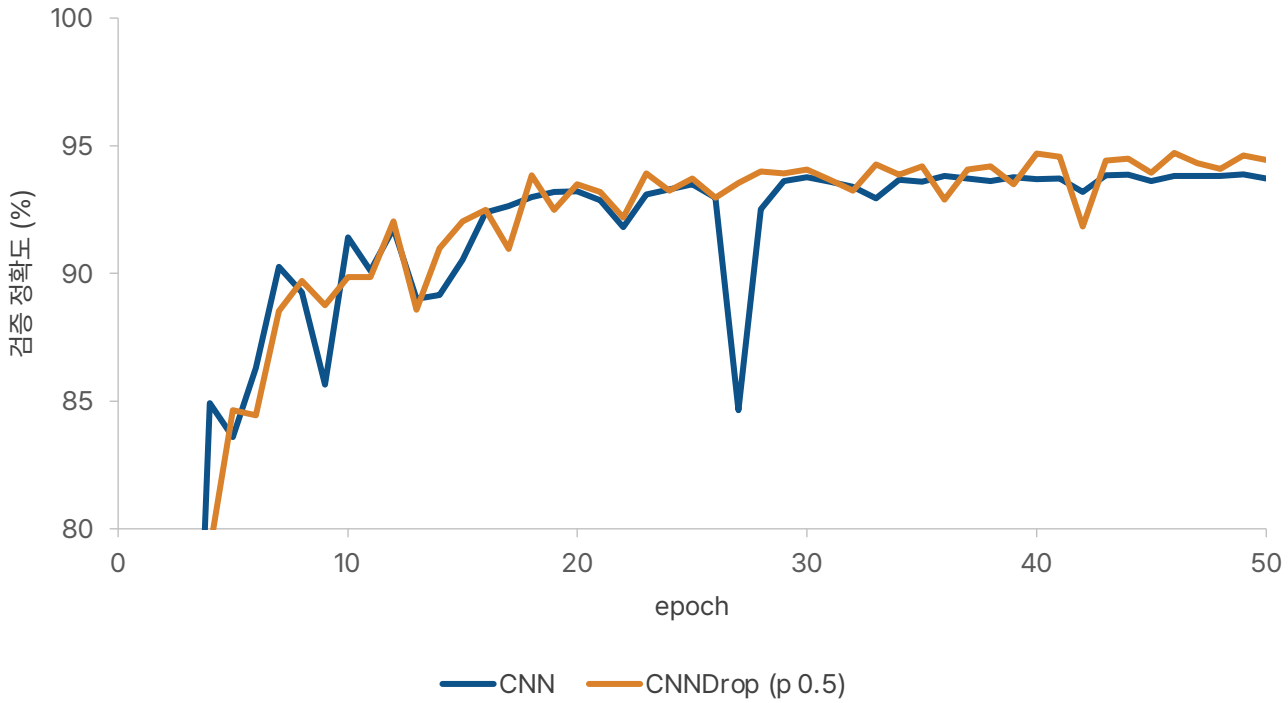
3. 실습 2: 정답 예시

실습 2. 정답 예시

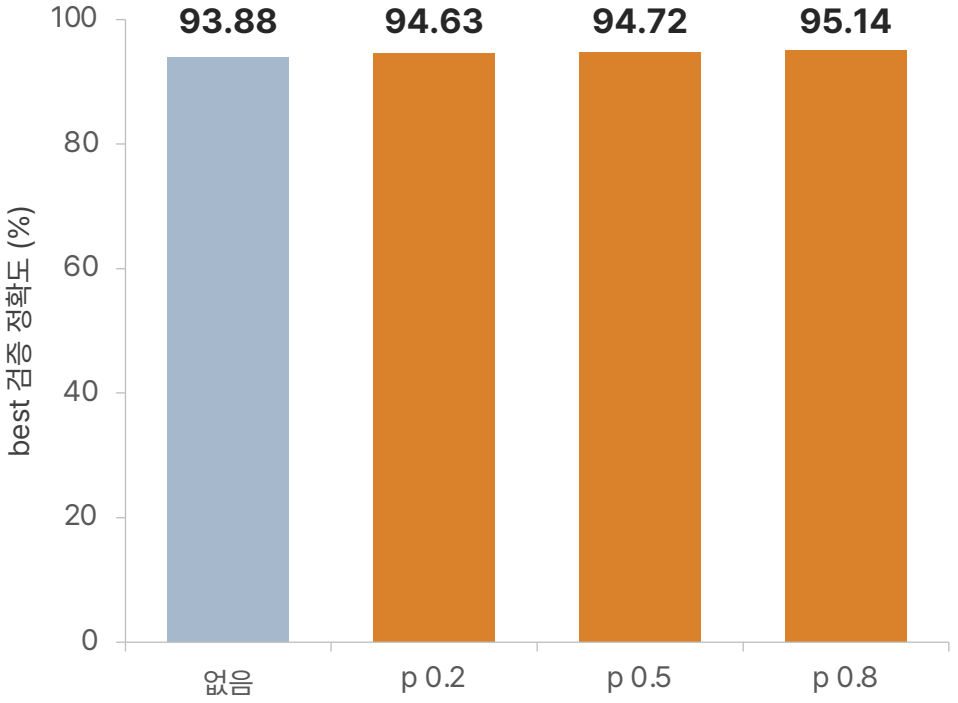
```
torch.manual_seed(0)                          # 예제 6-1과 같은 seed
model = CNNDrop().to(device)                  # 1. 모델 한 줄만 바꾼다
train_losses, val_losses, val_accs = fit(model, small_loader)
# → (예) best 46 94.72
train_loss, train_acc = evaluate(model, small_loader)    # 2. 학습 데이터 정확도
print(round(train_acc, 2), round(val_accs[-1], 2))   # → (예) 99.9 94.44
```

- **결과**
  - best 검증 정확도는 **94.72%**로, CNN의 93.88%보다 0.84%p 높다
  - 간격은 6.27%p에서 **5.46%p**로 줄었다

3. 실습 2: 결과, Dropout 비율에 따라



1,000장, 50 epoch, epoch별 검증 정확도 (실측)



Dropout 비율만 바꾼 실측

- Dropout을 넣으면 끝까지 조금씩 올라 CNN을 넘는다, 비율 사이 차이는 seed 흔들림보다 작아서 seed 한 번으로는 **고를 수 없다**

3. 예제 문제, 평가 모드를 잊으면

문제

1. 실습 2의 CNNDrop을 `model.eval()` 없이(학습 모드 그대로) 평가하면 검증 정확도는 오를까, 내릴까?
2. 학습 모드 그대로 같은 평가를 두 번 하면 같은 값이 나올까?
3. 직접 쓴 학습 루프에서 `model.train()`을 빼고, 검증 함수 뒤에 계속 학습하면 Dropout은 어떻게 될까?

3. 예제 문제 풀이

1과 2를 직접 재 보면

```
def evaluate_no_eval(model, loader):   # model.eval()만 뺀 검증
    model.train()                          # 학습 모드 그대로 평가
    correct = 0
    with torch.no_grad():
        for x, y in loader:
            out = model(x.to(device))
            correct += (out.argmax(dim=1).cpu() == y).sum().item()
    return correct / len(loader.dataset) * 100

print(round(evaluate(model, val_loader)[1], 2))        # → (예) 94.44   (평가 모드)
print(round(evaluate_no_eval(model, val_loader), 2))   # → (예) 91.89   (문제 1)
print(round(evaluate_no_eval(model, val_loader), 2))   # → (예) 91.36   (문제 2)
```

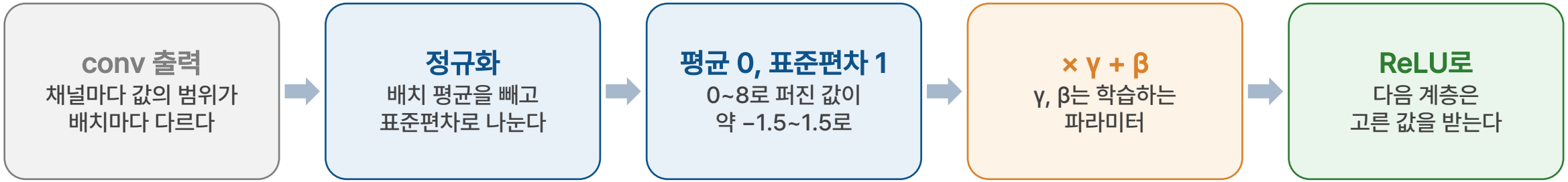
문제	답
1	**내린다**, 94.44% → 91.89%
2	**다르다**, 91.89%와 91.36%
3	뉴런을 모두 켠 채(Dropout 없이) 학습한다, 오류 없이 효과만 사라진다

- 그래서 표준 틀은 학습 함수 첫 줄에 `model.train()`, 검증 함수 첫 줄에 `model.eval()`을 둔다

PART 04

BatchNorm

4. BatchNorm의 원리



학습 모드 model.train()
지금 배치의 평균과 분산으로 정규화하고, 누적 평균과 분산을 조금씩 (10%) 갱신

평가 모드 model.eval()
학습 중에 모은 누적 평균과 분산(running_mean, running_var)으로 정규화

정규화 계산

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad y = \gamma\,\hat{x} + \beta$$

x는 conv 출력, x̂은 정규화한 값, y는 BN의 출력, μ와 σ²은 채널마다 구한 배치의 평균과 분산(σ²은 표준편차의 제곱, 평가 모드에서는 누적 값), ε은 0으로 나누지 않게 더하는 아주 작은 수

- **BatchNorm**(줄여서 BN)은 채널마다 배치의 평균을 빼고 **표준편차**(값이 퍼진 정도)로 나눠 범위를 맞춘 뒤, 학습하는 **γ, β**로 다시 조절한다

4. 예제 6-4: BatchNorm2d가 하는 일

예제 6-4. 평균 약 5, 표준편차 약 3인 배치를 BatchNorm2d에

```
import torch
import torch.nn as nn
torch.manual_seed(0)
bn = nn.BatchNorm2d(3)                          # 채널 3개를 정규화
x = torch.randn(8, 3, 4, 4) * 3 + 5             # 평균 약 5, 표준편차 약 3인 배치
out = bn(x)                                     # 학습 모드, 이 배치의 평균과 분산으로
print(round(out.mean().item(), 2), round(out.std().item(), 2))   # → 0.0 1.0
print(bn.running_mean)                          # → (예) tensor([0.5124, 0.5254, 0.4999])   (누적 평균은 10%씩)
bn.eval()                                       # 평가 모드, 누적 평균과 분산으로
print(round(bn(x).mean().item(), 2))            # → (예) 3.51   (아직 0이 아니다)
```

누적 평균 갱신

$$\mu_{\text{run}} \leftarrow 0.9\,\mu_{\text{run}} + 0.1\,\mu_{\text{batch}}$$

μ_{run}은 누적 평균, μ_{batch}는 배치 평균, ←는 왼쪽을 오른쪽 값으로 바꾼다는 뜻, 첫 채널은 처음 0에서 0.9 × 0 + 0.1 × 5.12 ≈ 0.51 (배치 평균 5.12)

- 처음에는 γ = 1, β = 0이라 학습 모드 출력이 **평균 0, 표준편차 1**이고, 누적 평균은 한 번에 **10%씩** 옮겨 간다
- 평가 모드는 누적 값을 쓴다, 지금은 한 번만 쌓여 평균이 0이 아니다

4. 예제 6-5: CNN에 BatchNorm 넣기

예제 6-5. CNN을 복사해 네 곳 변경

```
class CNNBN(nn.Module):                      # CNN을 복사해 네 곳을 바꾼다
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)       # 바뀐 곳 1, 채널 16
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)       # 바뀐 곳 2, 채널 32
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x):
        x = self.pool(self.act(self.bn1(self.conv1(x))))   # 바뀐 곳 3
        x = self.pool(self.act(self.bn2(self.conv2(x))))   # 바뀐 곳 4
        return self.fc(self.flat(x))

print(sum(p.numel() for p in CNNBN().parameters()))   # → 20586   (96개 늘었다)
```

- **바뀐 곳**
  - conv마다 `nn.BatchNorm2d(출력 채널 수)`를 하나씩 두고, **conv와 ReLU 사이**에 넣는다
- **확인**
  - γ, β가 채널마다 하나씩이라 16 × 2 + 32 × 2 = **96개**가 늘어 20,586개다

4. 실습 3: 1,000장에서 BatchNorm (문제)

실습

1. 예제 6-1을 복사해 모델만 `CNNBN()`으로 바꿔 1,000장으로 학습하기
2. best 검증 정확도와 간격을 실습 2의 표에 한 줄 추가하기
3. 누적 평균 갱신 횟수(`model.bn1.num_batches_tracked`)를 먼저 예측한 뒤, `state_dict()`의 이름과 함께 출력하기

- **힌트**
  - 이름 목록은 `list(model.state_dict().keys())`로 본다, 모델 파일에 저장하는 그 묶음이다
  - 누적 평균은 학습 배치 하나마다 한 번 갱신된다, 1,000장은 epoch마다 배치 16개다
- **생각해 볼 것**
  - MLP의 `state_dict()`는 이름이 4개였다, BatchNorm이 있으면 무엇이 더 저장될까?

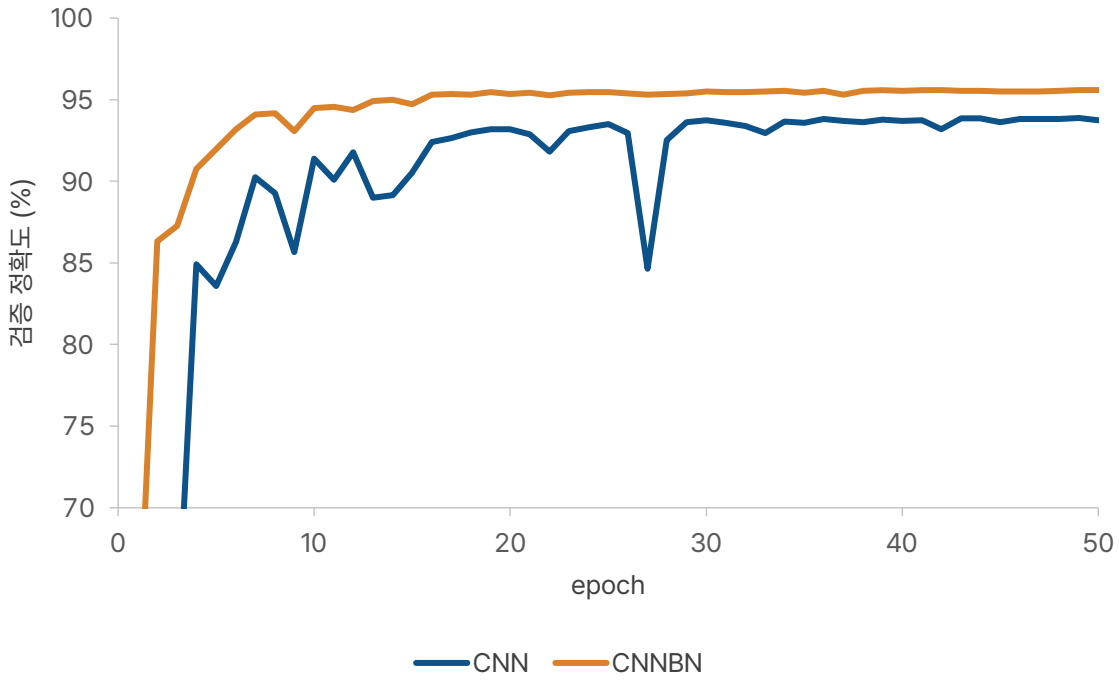
4. 실습 3: 정답 예시

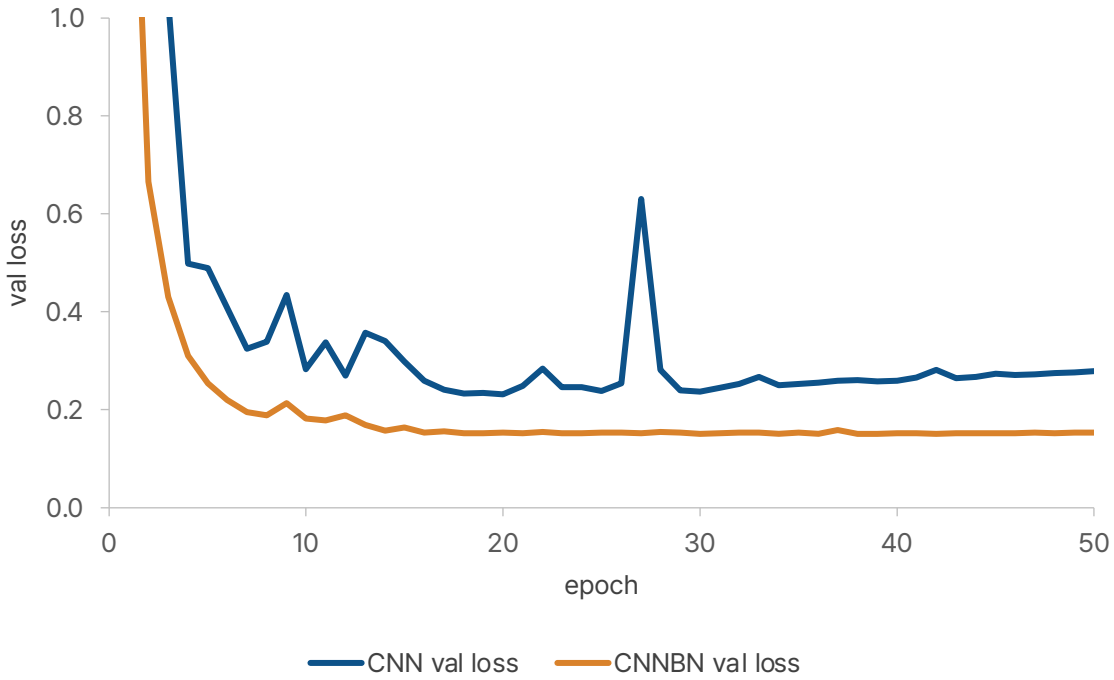
실습 3. 정답 예시

```
torch.manual_seed(0)
model = CNNBN().to(device)                    # 1. 모델 한 줄만 바꾼다
train_losses, val_losses, val_accs = fit(model, small_loader)
# → (예) best 42 95.59
train_loss, train_acc = evaluate(model, small_loader)    # 2. 학습 데이터 정확도
print(round(train_acc, 2), round(val_accs[-1], 2))   # → (예) 100.0 95.59
names = list(model.state_dict().keys())    # 3. 파일에 저장되는 이름들
print(len(names))                          # → 16
print(names[4:7])
# → ['bn1.running_mean', 'bn1.running_var', 'bn1.num_batches_tracked']
print(model.bn1.num_batches_tracked.item())   # → 800   (16 배치 × 50 epoch)
```

- **결과**
  - best 검증 정확도는 **95.59%**로 CNN보다 1.71%p 높고, 간격은 4.41%p다
- **3. state_dict**
  - BN마다 weight(γ), bias(β)와 **running_mean, running_var, num_batches_tracked**가 저장된다
  - 누적 평균은 16배치 × 50 epoch = **800번** 갱신됐다

4. 실습 3: 결과, BatchNorm은 빠르고 안정되게 배운다





1,000장, 50 epoch, 모델만 바꾼 실측

- 검증 93%에 **6 epoch** 만에 닿았다, CNN은 18 epoch이 걸렸다
- CNN의 val loss는 다시 오르는데 BN은 거의 오르지 않았고, 간격도 6.27%p에서 **4.41%p**로 줄었다 (이 실험의 실측)

4. Dropout과 BatchNorm, 학습 모드와 평가 모드

	학습 모드 `model.train()`	평가 모드 `model.eval()`	파일에 저장되는 것
Dropout	뉴런 일부를 무작위로 끄고, 남은 값을 키운다	모두 켠다, 결과가 늘 같다	없음 (파라미터 없음)
BatchNorm	지금 배치의 평균과 분산으로 정규화, 누적 값 갱신	누적 평균과 분산으로 정규화	γ, β와 누적 평균, 분산, 갱신 횟수
그 밖의 계층 (conv, fc, ReLU)	같다	같다	conv와 fc의 weight, bias

- 그래서 학습 함수 첫 줄의 `model.train()`과 검증 함수 첫 줄의 `model.eval()`은 **빼면 안 된다**
- 지금까지 `model.eval()`을 빼도 결과가 같았던 것은 MLP와 CNN에 이 두 계층이 없었기 때문이다

4. 예제 문제, BatchNorm 진단

문제

1. 실습 3의 CNNBN을 `model.eval()` 없이(학습 모드 그대로) 검증하면 정확도와 누적 평균은 어떻게 될까?
2. 학습할 때 배치를 64장에서 2장으로 줄이면 BatchNorm의 정규화는 믿을 만할까?
3. CNNBN의 파라미터는 20,586개다, running_mean과 running_var가 이 개수에 들어 있지 않은 이유는?

4. 예제 문제 풀이

문제	풀이	근거
1	검증 배치의 통계로 정규화하고 **누적 평균까지 바뀐다**, 검증 데이터가 모델 상태를 바꾸는 것이 문제다	평가 모드 95.59%, 학습 모드 95.58% (실측)
2	흔들릴 수 있다, 통계가 그 두 장의 사진에 크게 좌우된다	첫 BN은 채널마다 2 × 28 × 28개 값을 쓰지만 사진은 2장, BN은 보통 배치가 클 때 쓴다
3	기울기로 학습하는 값이 아니라 평균과 분산을 모아 두는 **버퍼**라서다	parameters()에는 없고 state_dict에는 있다

- 평가할 때는 늘 `model.eval()`로 누적 값을 쓴다, 그래야 검증 결과가 배치 구성에 따라 흔들리지 않는다

PART 05

데이터 증강

5. 데이터 증강과 그 종류

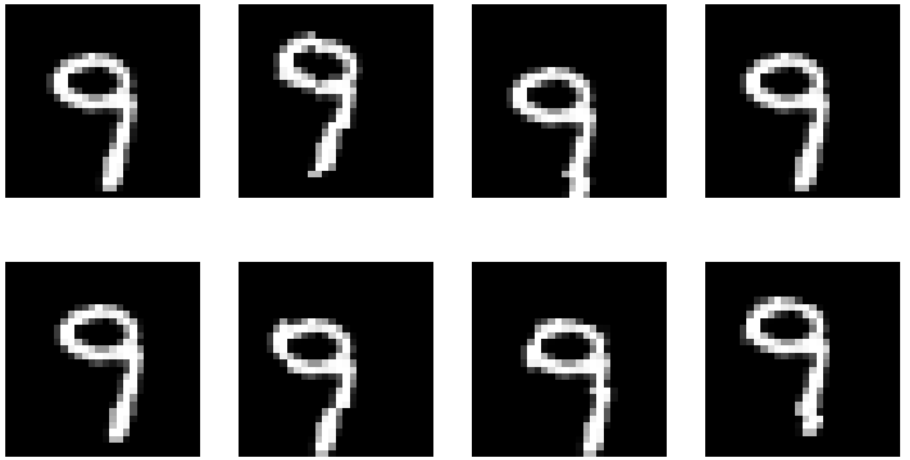
변환 (torchvision)	하는 일	숫자 사진 (MNIST)	고양이, 자동차 사진
`RandomCrop(28, padding=2)`	테두리를 2칸 늘린 뒤 28×28을 무작위로 잘라 조금 옮긴다	**괜찮다**	괜찮다
`RandomRotation(10)`	±10도 안에서 무작위로 돌린다	**괜찮다**	괜찮다
`RandomHorizontalFlip()`	절반의 확률로 좌우를 뒤집는다	**안 된다**, 2, 3, 7의 모양이 바뀐다	괜찮다
180도 회전	위아래를 거꾸로 돌린다	**안 된다**, 6과 9가 바뀐다	대부분 어색하다
`ColorJitter`	밝기, 대비, 색을 조금 바꾼다	흑백이라 색은 안 바뀌고 밝기와 대비만 바뀐다	괜찮다

- **데이터 증강**은 학습 사진을 조금씩 바꿔 매 epoch 다른 사진처럼 보여 준다, 바꿔도 **정답이 그대로**여야 한다
- 사진 수는 늘지 않는다, 같은 사진이 epoch마다 **모습만** 바뀐다

5. 예제 6-6: 증강한 사진 보기

예제 6-6. 같은 사진을 일곱 번 증강

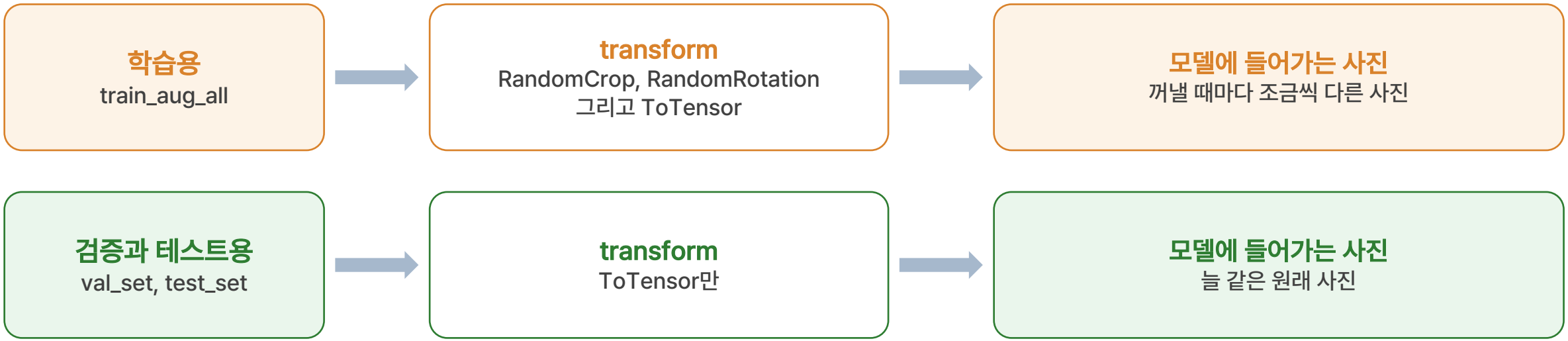
```
import matplotlib.pyplot as plt
raw = datasets.MNIST(root="data", train=True, download=True)   # PIL 사진 그대로
img, label = raw[small_idx[0]]                    # 학습 데이터 첫 사진
aug = transforms.Compose([transforms.RandomCrop(28, padding=2),   # 이동
                          transforms.RandomRotation(10)])         # 회전
torch.manual_seed(0)
fig, axes = plt.subplots(2, 4, figsize=(6, 3.2))
axes = axes.ravel()                               # 8칸을 한 줄로
axes[0].imshow(img, cmap="gray")                  # 원본
for i in range(1, 8):                             # 같은 사진을 7번 증강
    axes[i].imshow(aug(img), cmap="gray")
for ax in axes:
    ax.axis("off")
plt.show()
print(label)                                      # → (예) 9
```



왼쪽 위가 원본, 나머지 일곱 장이 증강 결과 (실행 결과)

- 매번 조금씩 옮겨지고 돌아가지만 **같은 숫자**다, 모델은 epoch마다 새 사진을 보는 셈이라 외우기 어렵다

5. 증강은 학습 데이터에만



학습 사진은 train_all과 train_aug_all에서 같은 번호(small_idx)로 고르고, 검증은 train_all의 val_set 그대로 쓴다

- 검증과 테스트는 **원래 사진**으로 잰다, 증강하면 평가할 때마다 사진이 달라져 점수가 흔들린다
- 그래서 MNIST 학습 파일을 transform만 다르게 **두 번** 연다

5. 예제 6-7: 증강용 데이터셋 만들기

예제 6-7. transform만 바꿔 다시 열고, 같은 번호로 고르기

```
train_tf = transforms.Compose([transforms.RandomCrop(28, padding=2),
                               transforms.RandomRotation(10),
                               transforms.ToTensor()])          # 증강한 뒤 Tensor로
train_aug_all = datasets.MNIST(root="data", train=True, download=True,
                               transform=train_tf)              # 같은 MNIST, transform만 다르다
small_aug_loader = DataLoader(Subset(train_aug_all, small_idx), batch_size=64, shuffle=True)
x, y = next(iter(small_aug_loader))           # 꺼낼 때마다 새로 증강된다
print(x.shape, len(small_aug_loader.dataset))   # → torch.Size([64, 1, 28, 28]) 1000
```

- 증강은 PIL 사진에 하고 **마지막에** `ToTensor()`로 바꾼다
- 같은 `small_idx`로 고르니 사진은 small_loader와 같고, 꺼낼 때마다 **새로 증강**된다

5. 실습 4: 1,000장에 증강 적용 (문제)

실습

1. 예제 6-1을 복사해 로더만 `small_aug_loader`로 바꿔 학습하기
2. best 검증 정확도와 간격을 표에 한 줄 추가하기
3. 좌우 뒤집기만 하는 새 로더(`RandomHorizontalFlip()`과 `ToTensor()`)로 학습해 비교하기

- **힌트**
  - 모델은 `CNN()` 그대로, `fit(model, small_aug_loader)`처럼 **로더만** 바꾼다
  - 학습 데이터 정확도는 증강하지 않은 `small_loader`로 잰다, 증강한 사진으로 재면 매번 달라진다
  - 3은 **새 이름**(`flip_tf`, `flip_all`, `flip_loader`)으로 만든다, 예제 6-7의 이름을 고쳐 쓰면 실습 5와 예제 6-10이 바뀐다

5. 실습 4: 정답 예시

실습 4. 정답 예시, 1과 2

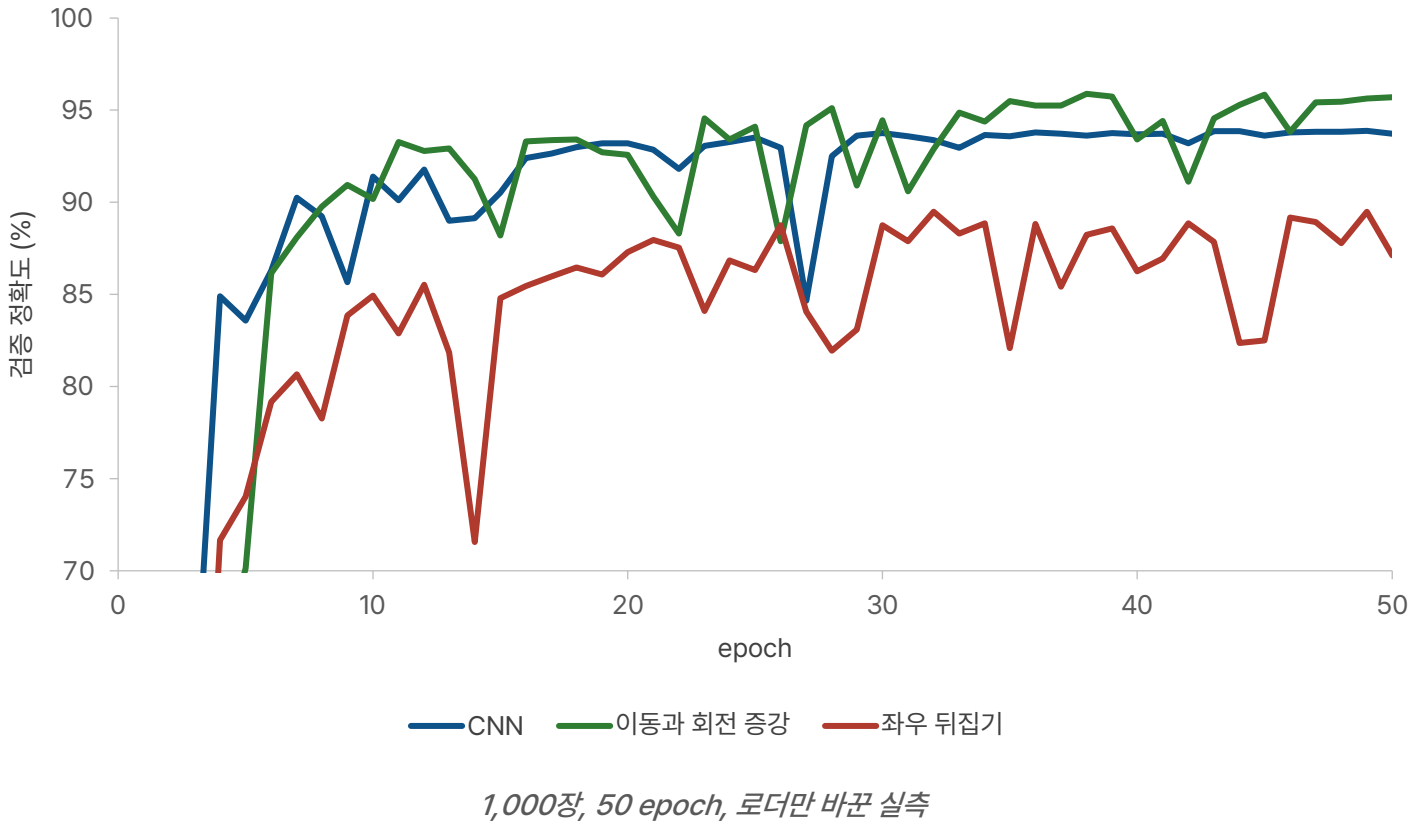
```
torch.manual_seed(0)
model = CNN().to(device)                          # 모델은 기본 CNN 그대로
train_losses, val_losses, val_accs = fit(model, small_aug_loader)   # 1. 로더만 바꾼다
# → (예) best 38 95.89
train_loss, train_acc = evaluate(model, small_loader)   # 2. 증강하지 않은 원래 1,000장으로
print(round(train_acc, 2), round(val_accs[-1], 2))      # → (예) 98.3 95.72
```

3. 좌우 뒤집기, 새 이름으로 만든다

```
flip_tf = transforms.Compose([transforms.RandomHorizontalFlip(),   # 3. 좌우 뒤집기
                              transforms.ToTensor()])
flip_all = datasets.MNIST(root="data", train=True, download=True,
                          transform=flip_tf)          # 새 이름, 예제 6-7은 그대로 둔다
flip_loader = DataLoader(Subset(flip_all, small_idx), batch_size=64, shuffle=True)
torch.manual_seed(0)
model = CNN().to(device)
train_losses, val_losses, val_accs = fit(model, flip_loader)
# → (예) best 32 89.49   (기본 CNN보다 낮다)
```

- **결과**
  - best 검증 정확도는 **95.89%**로 CNN보다 2.01%p 높고, 간격은 2.58%p다
  - 학습 데이터 정확도는 98.3%로 100%가 아니다, 매 epoch 새 사진을 보니 **다 외우지 못한다**
  - 3은 새 이름으로 만들어 `small_aug_loader`를 **그대로 둔다**

5. 실습 4: 결과, 맞는 증강과 틀린 증강



1,000장, 50 epoch, 로더만 바꾼 실측

로더	best 검증 정확도
원래 사진	93.88%
이동과 회전	**95.89%**
좌우 뒤집기	89.49%

- 이동과 회전은 검증 정확도를 올렸고, 좌우 뒤집기는 **오히려 떨어뜨렸다**
- 뒤집힌 2, 3, 7처럼 실제로는 없는 모양까지 같은 숫자라고 가르친 셈이다

5. 예제 문제, 이 증강은 괜찮을까

문제

1. 설치 방향이 정해진 부품 사진으로 정상과 불량을 가린다, 불량은 늘 오른쪽 위 모서리의 흠집이다, 좌우 뒤집기는 괜찮을까?
2. 예제 6-1의 CNN(원래 검증 사진 93.73%)을 증강한 검증 사진(예제 6-7의 이동과 회전)으로 재면 정확도는 어떻게 될까, 얼마나 바뀔까?
3. 예제 6-7의 `RandomRotation(10)`을 `RandomRotation(90)`으로 키우면 검증 정확도는 오를까, 내릴까?

5. 예제 문제 풀이

문제	풀이	근거
1	**안 된다**, 뒤집으면 흠집이 왼쪽 위로 가는데 그런 불량은 실제로 없다	정답이 그대로인지는 데이터를 아는 사람이 판단한다
2	**크게 떨어지고**(73% 근처), 잴 때마다 **흔들린다**, 증강 없이 학습한 모델이라 특히 크다	원래 93.73%, 증강 73.88%와 73.33%, 실습 4 모델도 92.19%와 92.28% (실측)
3	**내린다**, 90도까지 돌린 숫자는 실제 손글씨에 없는 모양이다	best 89.45%, 10도는 95.89%, 증강 없이 93.88% (실측)



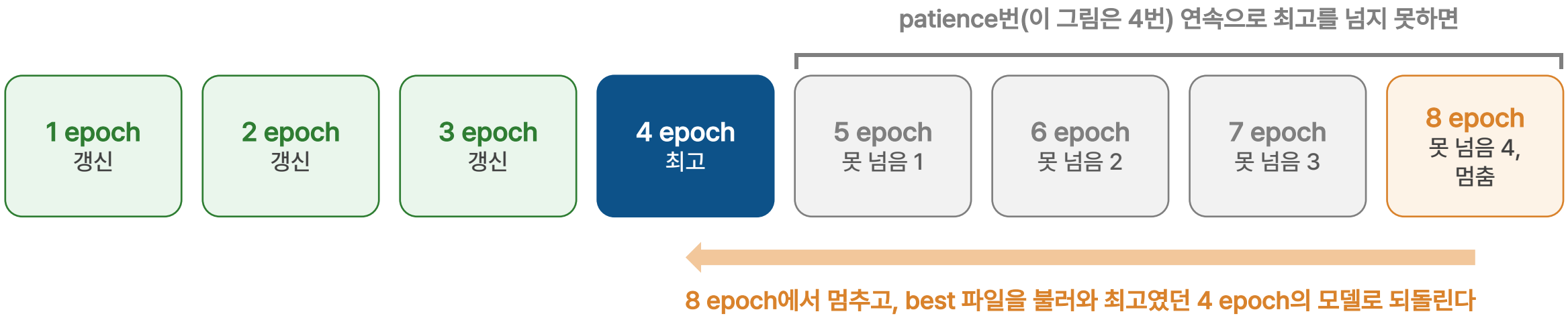
윗줄은 학습 데이터, 아랫줄은 RandomRotation(90)으로 돌린 것

- 증강을 고르는 기준은 '많이'가 아니라 **'정답이 그대로이고 실제로 나올 법한가'**다

PART 06

Early Stopping과 종합 실험

6. Early Stopping



멈추는 epoch $e_{\text{stop}} = e_{\text{best}} + \text{patience}$ e_{best}는 최고 기록이 나온 epoch, 최대 epoch 전에 멈출 때 성립, 그림은 4 + 4 = 8

- 최고 기록 뒤 **patience번 연속** 최고를 넘지 못하면 그 epoch에서 멈추고, 최고였던 모델로 되돌린다
- best 모델 저장에 **멈추기**를 더한 것이다, 기준(검증 정확도나 val loss)은 미리 하나로 정한다

6. 예제 6-8: patience로 멈추는 fit_es

예제 6-8. fit에 Early Stopping을 더한 함수

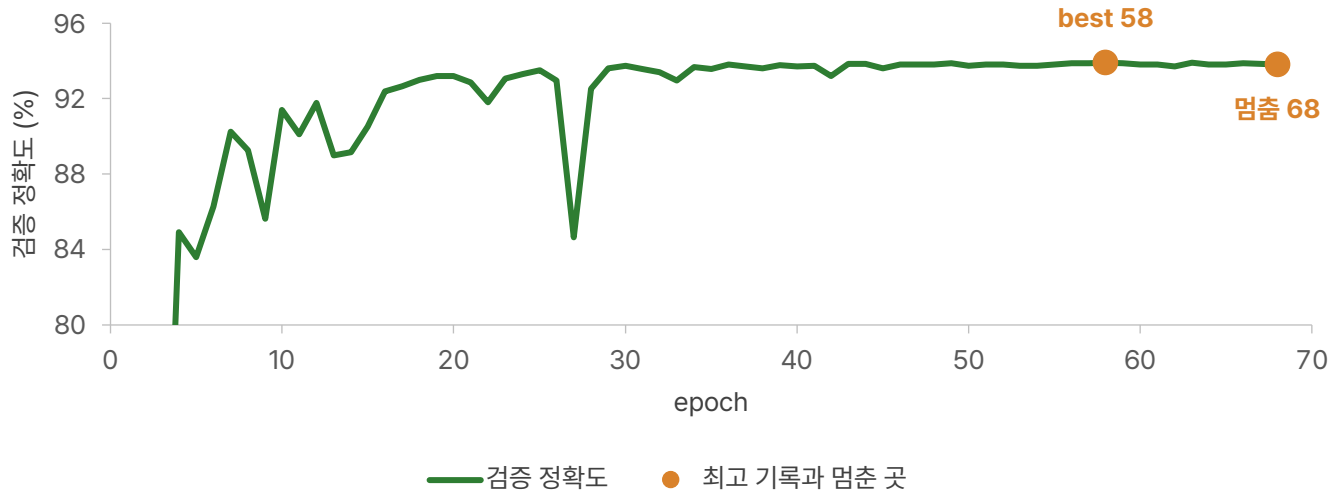
```
def fit_es(model, loader, patience=10, max_epochs=100):   # fit에 Early Stopping을 더했다
    opt = torch.optim.SGD(model.parameters(), lr=0.1)
    best_acc, best_epoch, val_accs = 0.0, 0, []
    for epoch in range(max_epochs):
        train_one_epoch(model, loader, opt)
        vl, va = evaluate(model, val_loader)
        val_accs.append(va)
        if va > best_acc:                          # 최고 기록이면 저장
            best_acc, best_epoch = va, epoch + 1
            torch.save(model.state_dict(), "best_es.pt")
        elif epoch + 1 - best_epoch >= patience:   # patience번 연속 못 넘으면 멈춘다
            break
    print("stop", epoch + 1, "best", best_epoch, round(best_acc, 2))
    model.load_state_dict(torch.load("best_es.pt", map_location=device))   # best로 되돌린다
    return val_accs
```

- **바뀐 곳**
  - 최고 기록이면 **파일로 저장**한다
  - 최고 기록 뒤 **patience번 연속** 못 넘으면(10번째) `break`로 멈춘다, 같은 값은 넘은 것이 아니다
  - 끝나면 best 파일을 불러와 **최고였던 모델로 되돌린다**

6. 예제 6-9: Early Stopping 실행

예제 6-9. 1,000장, patience 10, 최대 100 epoch

```
torch.manual_seed(0)
model = CNN().to(device)
val_accs = fit_es(model, small_loader)          # 1,000장, 최대 100 epoch
# → (예) stop 68 best 58 93.91
print(round(evaluate(model, val_loader)[1], 2))   # → (예) 93.91   (best 모델로 되돌아왔다)
```



예제 6-9의 기록 (실측)

- 58 epoch에 최고 **93.91%**를 찍고, 10 epoch 동안 넘지 못해 **68 epoch에서 멈췄다**
- 다시 재면 best와 같다, 최고였던 모델로 되돌아왔다
- val loss 기준이면 30 epoch에 멈추고 20 epoch 모델(93.21%)을 고른다, 그래서 기준은 **미리 하나로** 정한다

6. 실습 5: 세 가지를 합친 모델 (문제)

실습

1. CNNBN과 CNNDrop의 바뀐 곳을 합쳐 `CNNAll` 클래스 만들기
2. 합친 모델 `CNNAll`을 증강한 1,000장(`small_aug_loader`)으로 학습해 표에 한 줄 추가하기
3. 학습 함수를 `fit` 대신 `fit_es`로 바꿔 멈추는 epoch 보기

- **힌트**
  - CNNBN을 복사해 Dropout 두 곳(예제 6-3의 바뀐 곳 1, 2)을 더하면 된다
  - 학습은 `fit(model, small_aug_loader)`로 하고, 학습 데이터 정확도는 `small_loader`로 잰다
  - Dropout 비율은 **0.5**로 둔다, 실습 2의 셋째에서 바꿨다면 되돌린다
- **예측**
  - BN, Dropout, 증강 세 가지를 합치면 하나씩 쓸 때보다 더 좋아질까?

6. 실습 5: 정답 예시, CNNAll 클래스

실습 5. 정답 예시, 1. CNNAll

```
class CNNAll(nn.Module):                    # CNNBN과 CNNDrop의 바뀐 곳을 합친다
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.drop = nn.Dropout(0.5)
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x):
        x = self.pool(self.act(self.bn1(self.conv1(x))))
        x = self.pool(self.act(self.bn2(self.conv2(x))))
        return self.fc(self.drop(self.flat(x)))
```

- **합친 곳**
  - CNNBN의 BatchNorm 두 개에, 예제 6-3의 **Dropout 두 곳**을 더했다
  - 파라미터는 CNNBN과 같은 **20,586**개다, Dropout에는 파라미터가 없다

6. 실습 5: 정답 예시, 학습과 fit_es

실습 5. 정답 예시, 2. 증강한 1,000장으로 학습

```
torch.manual_seed(0)
model = CNNAll().to(device)                    # 2. BN과 Dropout을 넣은 모델에
train_losses, val_losses, val_accs = fit(model, small_aug_loader)   # 증강한 1,000장
# → (예) best 45 97.39
train_loss, train_acc = evaluate(model, small_loader)
print(round(train_acc, 2), round(val_accs[-1], 2))   # → (예) 99.3 96.55
```

3. fit 대신 fit_es

```
torch.manual_seed(0)
model = CNNAll().to(device)
val_accs = fit_es(model, small_aug_loader)     # 3. fit 대신 fit_es, 증강한 1,000장
# → (예) stop 62 best 52 97.43
```

- **결과**
  - best 검증 정확도는 **97.39%**로 CNN보다 3.51%p 높고, **하나씩 쓸 때보다** 높다
  - 학습 데이터 정확도는 99.3%이고, 간격은 2.75%p다
  - 3. `fit_es`는 **62 epoch**에서 멈추고 52 epoch 모델(97.43%)로 되돌아온다

6. 1,000장 실험 기록표

실험 (1,000장, 50 epoch)	바꾼 것	best 검증 정확도 (seed 0)	seed 0~2 범위	CNN 대비 (seed 0)
기본	CNN	93.88%	93.88~94.01%	기준
Dropout	모델, CNNDrop	94.72%	94.72~95.45%	+0.84%p
BatchNorm	모델, CNNBN	95.59%	95.41~95.59%	+1.71%p
증강	로더, 이동과 회전	95.89%	95.89~96.41%	+2.01%p
세 가지 모두	CNNAll, 증강 로더	**97.39%**	97.39~97.62%	**+3.51%p**

같은 1,000장, 같은 50 epoch, 바꾼 것 하나씩, 마지막 줄은 셋을 함께 (실측)

- 세 도구 모두 seed 세 번 내내 기본 CNN보다 높고, 증강이 가장 크며 **합쳤을 때** 가장 높다
- Dropout과 BatchNorm은 범위가 겹쳐 seed에 따라 순서가 바뀐다, 한 번의 결과로 순위를 말하지 않는다

6. 예제 6-10: 전체 데이터로 실험 4

예제 6-10. 지난시간 CNN 학습에서 모델과 로더를 바꿔 10 epoch

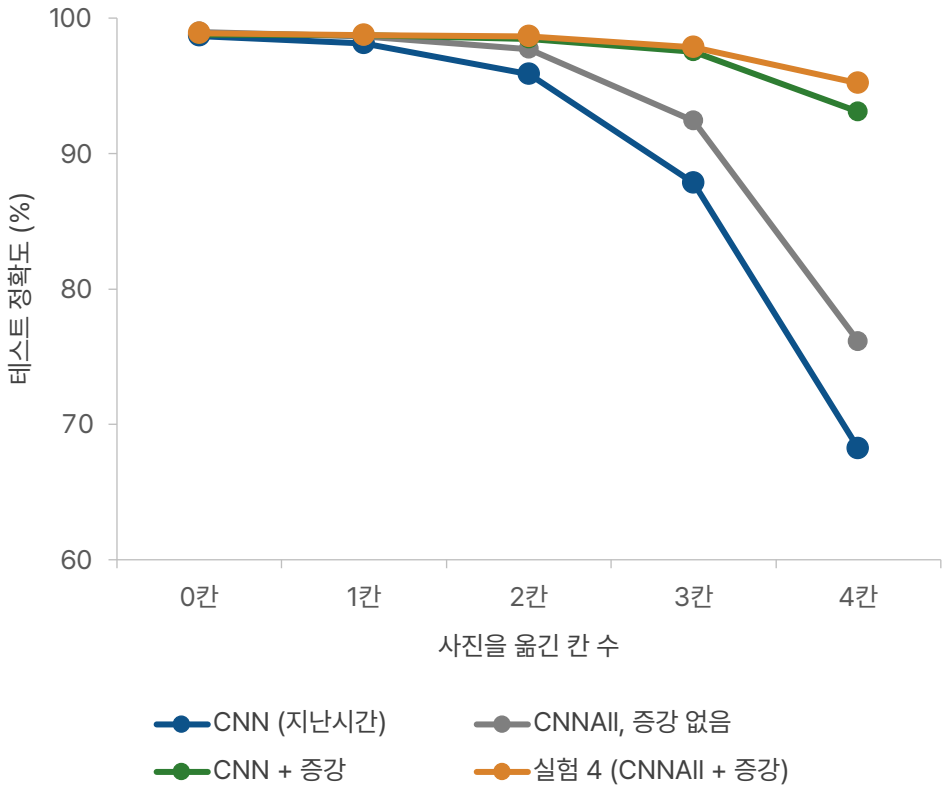
```
train_aug_loader = DataLoader(Subset(train_aug_all, train_set.indices),
                              batch_size=64, shuffle=True)    # 전체 50,000장, 증강
torch.manual_seed(0)                                  # 지난시간 CNN 학습과 같은 seed, 10 epoch
model = CNNAll().to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1)
best_acc, best_epoch = 0.0, 0
for epoch in range(10):
    train_one_epoch(model, train_aug_loader, opt)
    vl, va = evaluate(model, val_loader)
    print(epoch + 1, round(va, 2))                    # 진행 상황 보기
    if va > best_acc:
        best_acc, best_epoch = va, epoch + 1
        torch.save(model.state_dict(), "exp4_all.pt")   # 실험 4
print("best", best_epoch, round(best_acc, 2))   # → (예) best 10 98.77
```

- **결과**
  - best 검증 정확도는 **98.77%**(10 epoch)이고, 지난 시간 CNN은 98.54%였다
  - 마지막 epoch이 최고였다, 더 학습하면 **오를 여지**가 있다
- **바꾼 것**
  - 모델은 `CNNAll`, 로더는 전체 50,000장의 **증강 로더**다, 두 가지를 함께 바꿨다

6. 실험 4 결과: 정확도와 이동 견고성

실험	바꾼 것	best val (epoch)	test	Baseline 대비 (val)
실험 2	모델, MLP → CNN	98.54% (9)	98.67%	+1.29%p
실험 4	모델 CNNAll(BN, Dropout), 증강	**98.77%** (10)	**98.88%**	**+1.52%p**

- 원래 사진의 검증 정확도는 차이가 작다, 실험 4는 seed 세 번에 98.62~98.77%, 지난시간 CNN은 seed 다섯 번에 98.54~98.64%다
- 4칸 옮긴 사진에서 실험 4 모델은 **95.2%**로, CNN의 68.2%보다 훨씬 덜 떨어진다
- 증강만 넣은 CNN은 93.1%, 증강 없는 CNNAll은 76.1%였다, 이동에 강해진 것은 주로 **증강** 덕분이다 (추가 실측)



테스트 사진을 가로로 옮긴 실측 (회색과 초록은 추가 실측)

6. 예제 문제, 대책 고르기

문제

1. 학습 데이터 정확도 99%, 검증 80%다, 데이터는 더 모을 수 없다, 무엇부터 해 볼까?
2. 학습 데이터 정확도 70%, 검증 69%다, Dropout을 더 넣으면 좋아질까?
3. 검증 정확도가 20 epoch 근처에서 최고를 찍고 100 epoch까지 오르지 않는다, 무엇을 쓰면 좋을까?

6. 예제 문제 풀이

문제	진단	먼저 해 볼 것
1	간격 19%p, **과적합**	증강(정답이 그대로인 변환), Dropout, BatchNorm
2	간격 1%p라 과적합으로 보기 어렵다, **학습 부족**을 먼저 의심한다	Dropout은 학습을 더 어렵게 한다, 곡선과 데이터, 학습 설정부터 점검
3	20 epoch 이후는 시간 낭비	**Early Stopping**과 best 모델 저장

- 과적합 대책은 간격이 클 때 쓰는 도구다, 간격이 작은데 정확도도 낮으면 **배울 힘**부터 키운다

6. 정리, 다음 시간

오늘 배운 것	하는 일	주의
일반화 간격	학습 데이터 정확도 − 검증 정확도, 크면 과적합을 의심	판단은 늘 검증 데이터로, loss 곡선도 함께
Dropout	학습 중 뉴런 일부를 무작위로 꺼 외우기를 방해	평가 때는 `model.eval()`로 모두 켠다
BatchNorm	채널마다 배치의 범위를 맞춰 학습을 안정시킨다	평가 때는 누적 평균과 분산을 쓴다
데이터 증강	정답이 그대로인 변환으로 학습 사진을 매번 다르게 보여 준다	학습 데이터에만, 숫자에 뒤집기 금지
Early Stopping	검증이 나아지지 않으면 멈추고 best로 되돌린다	patience를 너무 작게 두지 않고, 기준은 미리 하나로

Dropout 비율과 patience 같은 값은 이후 실험으로 고른다

- **다음 시간에는 lr, batch, epoch, optimizer 같은 값을 실험으로 고르는 하이퍼파라미터 튜닝을 다룬다**